can retrieve the tokens associated with the specific user and validate the request.

However, session cookies are not enough to validate requests, which are vulnerable to Cross-Site Request Forgery (CSRF) attacks. CSRF affects *modifying* HTTP requests like `POST`, `PUT`, and `DELETE`. An attacker could induce users to perform requests they didn't intend by forging a request meant to cause harm. Forged requests could do things like transfer money from your bank account or compromise critical data.

WARNING Many online tutorials and guides show how to disable the CSRF protection as the first thing when configuring Spring Security. That's dangerous to do without explaining the reasoning or considering the consequences. I recommend keeping the protection enabled unless there's a good reason not to (you'll see one good reason in chapter 12). As a general guideline, browser-facing applications like Edge Service should be protected against CSRF attacks.

Fortunately, Spring Security has built-in protection against such attacks. The protection is based on a so-called CSRF token generated by the framework that's provided to the client at the beginning of the session and required to be sent along with any state-changing requests.

NOTE To learn more about CSRF protection in Spring Security, you can check out chapter 10 of *Spring Security in Action* by Laurențiu Spilcă (Manning, 2020), where the subject is explained in great detail.

In the previous section, you tried to log out, but the request failed. Since the logout operation is available through a `POST` request to the `/logout` endpoint, the application expects to receive the CSRF token generated by Spring Security for that user session. By default, the generated CSRF token is sent to the browser as an HTTP header. However, Angular applications cannot work with that and expect to receive the token value as a cookie. Spring Security supports this specific requirement, but it's not enabled by default.